CLAUDE

INHERITED FROM Helix Constitution

This module is a submodule of an ATMOSphere-family project that includes the Helix Constitution submodule at the parent's `constitution/` path. All rules in `constitution/CLAUDE.md` and the `constitution/Constitution.md` it references (universal anti-bluff covenant §11.4, no-guessing mandate §11.4.6, credentials-handling mandate §11.4.10, host-session safety §12, data safety §9, mutation- paired gates §1.1) apply unconditionally to every change landed here. The module-specific rules below extend them — they never weaken any universal clause.

When this file disagrees with the constitution submodule, the constitution wins. Locate the constitution submodule from any arbitrary nested depth using its `find_constitution.sh` helper.

Canonical reference: https://github.com/HelixDevelopment/Helix-Constitution

---

CLAUDE.md - HelixCode AI Agent Manual

INHERITED FROM HelixConstitution/CLAUDE.md

All rules in `HelixConstitution/CLAUDE.md` (and the `HelixConstitution/Constitution.md` it references) apply unconditionally. The project-specific rules below extend them. Rules below MUST NOT weaken any inherited clause.

HelixCode - AI Agent Operating Manual

Version: 1.0.0 **Date**: 2026-04-30 **Scope**: This document guides AI agents working on the HelixCode codebase **Authority**: Cascaded from HelixAgent root `CLAUDE.md` with HelixCode-specific addenda

---

1. Agent Identity & Purpose

You are an AI agent working on **HelixCode**, an enterprise-grade distributed AI development platform. Your work directly impacts the quality and usability of a production system.

Your mandate: Write real, working, tested code. No simulations. No placeholders. No "for now" implementations. Every feature you implement MUST actually work when a user invokes it.

1.1 Peer Governance Documents (keep in sync)

This `CLAUDE.md` sits alongside several other agent/governance manuals at the repo root. They overlap and must remain consistent:

- `CONSTITUTION.md` — source of truth for all mandates (CONST-033, CONST-035, CONST-036–040, Article XI §11.9). When this file conflicts with the Constitution, the Constitution wins.
- `AGENTS.md` — generic agent manual (40 KB; mirror anti-bluff rules here).
- `CRUSH.md`, `QWEN.md` — sibling agent manuals for other CLI tools. Cascade rule changes to all of them.
- `helix_code/CLAUDE.md`, `helix_qa/CLAUDE.md`, `challenges/CLAUDE.md` — submodule-scoped manuals; this root file inherits from them and they inherit from this one.

---

2. Universal Mandatory Rules (Non-Negotiable)

These rules cascade from the HelixCode Constitution. They are permanent and apply to every task.

Rule 1: No CI/CD Pipelines

No `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, `.travis.yml`, `.circleci/`, or any automated pipeline. All builds and tests run manually or via Makefile/script targets.

Rule 2: No Mocks in Production

Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. Only unit tests may use mocks.

Rule 3: No HTTPS for Git

SSH URLs only (`git@github.com:…`) for all Git operations.

Rule 4: No Manual Container Commands

Use the orchestrator binary (`make build` → `./bin/<app>`). Direct `docker`/`docker-compose` commands are prohibited as workflows.

Rule 5: Real Data for Non-Unit Tests

All integration, E2E, and challenge tests MUST use real infrastructure (real databases, real HTTP calls, real containers).

Rule 6: 100% Challenge Coverage

Every component MUST have Challenge scripts validating real-life use cases.

Rule 7: Reproduction-Before-Fix

Every bug MUST be reproduced by a Challenge script BEFORE any fix is attempted.

Rule 8: Definition of Done

A change is NOT done because code compiles. "Done" requires pasted terminal output from a real run against real artifacts.

Rule 9: No Self-Certification

Words like *verified, tested, working, complete, fixed, passing* are forbidden unless accompanied by pasted command output from that session.

Rule 10: Zero-Bluff Mandate (CONST-035)

A passing test is a claim that the feature **works for the end user**. Every test must guarantee Quality + Completion + Full Usability. Any test that doesn't certify all three is a bluff and must be tightened.

Constitutional anchors (cascaded from `CONSTITUTION.md`)

Article XI §11.9 — Anti-Bluff Forensic Anchor

> Verbatim user mandate: "*We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!*"
>
> Operative rule: **The bar for shipping is not "tests pass" but "users can use the feature."** Every PASS in this codebase MUST carry positive runtime evidence captured during execution. Metadata-only / configuration-only / absence-of-error / grep-based PASS without runtime evidence are critical defects regardless of how green the summary line looks. No false-success results are tolerable.

Article XII §12.1 (CONST-042) — No-Secret-Leak

No API key, token, password, certificate, or other credential may be committed to any repository owned by HelixDevelopment or vasic-digital. All secrets live in `.env` files (mode 0600) listed in `.gitignore`. Any leak is a release blocker until rotated and post-mortemed.

Article XII §12.2 (CONST-043) — No-Force-Push

No force push, force-with-lease push, history rewrite, branch deletion of `main`/`master`, or upstream-overwriting operation may be performed without explicit, in-conversation user approval per operation. Authorization for one push does not extend further. Bypassing hooks / signing / protected-branch rules also requires explicit approval.

---

3. HelixCode-Specific Architecture

3.1 Technology Stack

- **Language**: Go — root meta-repo on `go 1.25.2`, inner Go application (`helix_code/`) on `go 1.26`. Keep both modules current; do not downgrade.
- **Module IDs**: root `dev.helix.code` (thin), inner `dev.helix.code` (full app + transitive deps).
- **HTTP / API**: Gin v1.11.0, gorilla/websocket v1.5.3, gRPC v1.80.0.
- **Persistence**: PostgreSQL 15+ via pgx/v5 + lib/pq; Redis 7+ via go-redis/v9.
- **AuthN/Z**: golang-jwt/v4 v4.5.2, bcrypt/argon2 (`golang.org/x/crypto`), oauth2.
- **Config / CLI**: Viper v1.21.0, Cobra v1.8.0, pflag v1.0.10, fsnotify v1.9.0.
- **LLM / Cloud**: AWS Bedrock runtime (aws-sdk-go-v2), Azure azcore/azidentity, getzep/zep-go/v3, smacker/go-tree-sitter.
- **UI**: Fyne v2.7.0 (desktop GUI), tview / tcell/v2 (terminal UI), chromedp (headless browser).
- **Testing**: stretchr/testify v1.11.1.

3.2 Repository Layout — Meta-Repo + Submodules

This repo is a governance/meta-repo, not the Go application. The actual Go binary lives in the `helix_code/` subdirectory (a submodule). When an agent says "edit `internal/auth`," they almost always mean `helix_code/internal/auth`, not the root `internal/`.

```
helix_code/                                  # ← repo root
(governance + submodules)
├── CLAUDE.md / AGENTS.md / CONSTITUTION.md / CRUSH.md / QWEN.md
# agent manuals
├── Makefile                                 # governance gates only
(see §3.4)
├── go.mod                                   # thin root module
(dev.helix.code, go 1.25.2)
├── helix                                    # Docker facade script
(run platform standalone)
├── setup.sh                                 # one-shot: submodule
init + deps + build
├── .gitmodules                              # source of truth for
submodule wiring
├── docker-compose.helix.yml                 # standalone deployment
├── internal/{fix,security,testing,theme} # root-level helpers
ONLY (NOT the app)
├── cmd/security-test/                       # root-level security-
test tool ONLY
├── scripts/                                 # init-submodules,
```

```
propagate-governance,
│                                          #   verify-governance-
cascade, no-silent-skips,
│                                          #   demo-all, run-all-
tests, …
├── docs/                                  # ARCHITECTURE.md,
COMPLETE_*.md guides,
│                                          #   bluff-proofing/,
llms_verifier/, helix_qa/
│
├── helix_code/      ← TRACKED SUBDIRECTORY (NOT a submodule —
meta-repo's primary inner directory; circular reference if
promoted; see §3.2.1)
├── helix_qa/        ← SUBMODULE: QA / challenge-orchestration
platform
├── challenges/     ← SUBMODULE: cross-cutting Challenge bank
(Panoptic, banks/)
├── containers/     ← SUBMODULE: Docker/container artefacts
├── Dependencies/   ← SUBMODULES: LLama_CPP, Ollama,
HuggingFace_Hub, …
├── security/       ← SUBMODULE: security tooling
├── Assets/         ← SUBMODULE: logos, themes, brand
├── github_pages_website/ ← SUBMODULE: marketing site
└── Example_Projects/     ← reference projects (Aider, Cline,
Plandex, OpenHands, …)
```

3.2.1 Inner Go application — `helix_code/` submodule

```
helix_code/helix_code/                       # module
dev.helix.code, go 1.26
├── Makefile                                 # real build/test
targets (see §3.4)
├── cmd/
│   ├── server/                              # HTTP server entry →
bin/helixcode
│   ├── cli/                                 # CLI client entry →
bin/cli
│   ├── helix-config/                        # config tool
│   ├── config-test/                         # config validator
│   ├── security-test/, security-fix*/       # security tools
│   └── performance-optimization*/           # perf tools
├── internal/                                # ~45 packages — the
real domain code
│   ├── auth/        agent/      cognee/      commands/   config/
│   ├── context/     database/   deployment/  discovery/  editor/
│   ├── event/       focus/      hardware/    helixqa/    hooks/
│   ├── llm/         logging/    logo/        mcp/        memory/
│   ├── monitoring/  notification/ performance/ persistence/
project/
│   ├── provider/    providers/  redis/       repomap/    rules/
│   ├── security/    server/     session/     task/
```

```
template/
│   ├── tools/       verifier/   version/     worker/
workflow/
│   ├── adapters/    fix/        testutil/    mocks/      # mocks/
is unit-test-only
├── applications/
│   ├── desktop/      (Fyne GUI)
│   ├── terminal-ui/  (tview TUI)
│   ├── ios/  android/  aurora-os/  harmony-os/
├── tests/
│   ├── e2e/challenges/   # E2E challenge runner (cmd/runner/
main.go)
│   ├── integration/      # gated by `-tags=integration`
│   ├── unit/             # mocks ALLOWED here only
│   ├── security/         # security suite
│   └── performance/      # benchmarks
├── config/                # YAML configs (dev/, prod/, test/)
├── docker/  scripts/  shared/  qa-integration/
└── docker-compose.full-test.yml + .env.full-test    # zero-skip
integration stack
```

Cardinal rule: if a path in instructions doesn't start with `helix_code/`, `helix_qa/`, etc., assume it is relative to the inner Go module and prefix with `helix_code/`.

3.3 Historical Bluffs — Resolved, Guard Against Regression

The three patterns below were live bluffs in earlier revisions of `helix_code/cmd/cli/main.go`. They have been fixed (verify with `grep -rn "simulate\|For now\|TODO implement\|placeholder" helix_code/cmd/cli/main.go` — must return empty). Treat these as canonical anti-pattern examples; if a future change reintroduces any of them, the change is broken regardless of whether tests pass.

BLUFF-001: LLM Generation is Simulated

Location: `helix_code/cmd/cli/main.go` → function `handleGenerate`
Status: RESOLVED — now calls `provider.Generate` / `GenerateStream` directly. Do not regress. **Code Pattern**:

```
// ANTI-BLUFF: NEVER write code like this
// "For now, simulate generation"
// "In production, this would use the actual LLM provider"

// WRONG - SIMULATION:
response := fmt.Sprintf("Generated response for: %s\n\nThis is a
        simulated response...")

// CORRECT - REAL IMPLEMENTATION:
resp, err := c.llmProvider.Generate(ctx, req)
```

```
if err != nil {
    return fmt.Errorf("generation failed: %w", err)
}
fmt.Println(resp.Text)
```

Agent Rule: When implementing LLM-related code, you MUST make real HTTP calls to real providers. NEVER simulate responses.

3.4 Build & Test Commands

Two Makefiles. The **root** Makefile only runs governance gates; the **inner** `helix_code/Makefile` does real builds and tests. Always know which directory you are in.

Root governance gates (run from repo root):

```
make no-silent-skips          # fail on bare t.Skip() without SKIP-
        OK marker
make demo-all                 # run every submodule's demo (proves
        they actually run)
make demo-one MOD=<name>      # run one submodule's demo
make ci-validate-all          # all governance gates in warn-mode
./setup.sh                    # first-time: submodules + system
        deps + build
./scripts/init-submodules.sh                  # init all submodules
./scripts/propagate-governance.sh             # cascade
        Constitution/CLAUDE/AGENTS
./scripts/verify-governance-cascade.sh        # confirm anchors
        present in submodules
./helix start | stop | logs | shell           # Docker facade for
        the platform
```

Inner application (run from `helix_code/`):

```
make build                    # → bin/helixcode (server)
make verify-compile           # quick compile-only sanity check
make test                     # all unit tests
make test-coverage            # coverage with -race
make fmt                      # gofmt
make lint                     # golangci-lint run
make dev                      # build + run with config/dev/
        config.yaml
make prod                     # cross-compile linux/macos/windows
```

Full integration / E2E (real PostgreSQL + Redis + Ollama via docker-compose):

```
make test-infra-up                            # start docker-
        compose.full-test.yml
make test-infra-status                        # check stack health
make test-full                                # ALL tests, ZERO
        skips
```

```
make test-unit-full / test-integration-full / test-e2e-full /
        test-security-full
make test-verifier-unit / test-verifier-integration / test-
        verifier-challenges
make test-infra-down                          # tear down stack +
        volumes
```

Containerized builds (no host Go required):

```
make container-builder-image    # build the builder image once
make container-build            # build inside container
make container-test             # test inside container
make container-shell            # interactive shell in builder
make container-release          # full release in container
```

Single-test invocation (inner module):

```
cd HelixCode
go test -v -run TestJWTGenerate ./internal/
        auth                              # single unit test
go test -v -tags=integration -run TestAPI_CreateTask ./tests/
        integration/...
go test -v -count=1 ./internal/
        verifier/...                                  # disable test
        cache
go test -v -race -coverprofile=cover.out ./internal/
        llm                   # one pkg with race+cover
```

E2E challenges (real, end-to-end, runtime evidence required):

```
cd helix_code/tests/e2e/challenges && go run cmd/runner/main.go
        -all
# Or root-level cross-cutting Challenges:
cd Challenges && make <target>
```

Anti-bluff smoke check (must always pass):

```
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  helix_code/internal helix_code/cmd && echo "BLUFF FOUND" ||
        echo "clean"
```

Platform / mobile builds (inner module):

```
make desktop / desktop-nogui / desktop-linux / desktop-macos /
        desktop-windows
make mobile-init && make mobile-ios && make mobile-android
make aurora-os && make harmony-os
```

BLUFF-002: Model Listing is Hardcoded

Location: `helix_code/cmd/cli/main.go` → function `handleListModels`
Status: RESOLVED — must continue to query `c.providerManager.GetProviders()` per CONST-036/037 (LLMsVerifier is the single source of truth). **Correct Pattern**:

```
func (c *CLI) handleListModels(ctx context.Context) error {
    // Query ALL configured providers
    for name, provider := range c.providerManager.GetProviders() {
        models, err := provider.GetModels()
        if err != nil {
            log.Printf("Warning: failed to list models from %s:
        %v", name, err)
            continue
        }
        // Display real models
        for _, model := range models {
            fmt.Printf("%s/%s: %s (context: %d)\n", name,
        model.ID, model.Name, model.ContextSize)
        }
    }
    return nil
}
```

BLUFF-003: Command Execution is Simulated

Location: `helix_code/cmd/cli/main.go` → function `handleCommand` **Status**: RESOLVED — must continue to use `os/exec` via `exec.CommandContext` and surface real exit codes. Never replace with print-and-sleep. **Correct Pattern**:

```
func (c *CLI) handleCommand(ctx context.Context, command string)
        error {
    // ANTI-BLUFF: Actually execute the command
    cmd := exec.CommandContext(ctx, "sh", "-c", command)
    cmd.Dir = c.workingDirectory

    output, err := cmd.CombinedOutput()

    fmt.Printf("Exit code: %d\n", cmd.ProcessState.ExitCode())
    fmt.Printf("Output:\n%s\n", string(output))

    return err
}
```

---

4. Code Patterns for Agents

4.1 Interface-Driven Design

```
// Define the contract
type Provider interface {
    Generate(ctx context.Context, req *GenerateRequest)
        (*GenerateResponse, error)
    GetModels() ([]Model, error)
    HealthCheck(ctx context.Context) error
```

```
}

// Implement with REAL behavior
type OllamaProvider struct { ... }
func (p *OllamaProvider) Generate(ctx context.Context, req
        *GenerateRequest) (*GenerateResponse, error) {
    // Make REAL HTTP call
    // NO simulation
}
```

4.2 Manager Pattern

```
type TaskManager struct {
    db     TaskRepository
    mu     sync.RWMutex
    tasks  map[uuid.UUID]*Task
}

func (m *TaskManager) Create(ctx context.Context, task *Task)
        error {
    m.mu.Lock()
    defer m.mu.Unlock()

    // Persist to REAL database
    if err := m.db.Save(ctx, task); err != nil {
        return fmt.Errorf("failed to save task: %w", err)
    }

    m.tasks[task.ID] = task
    return nil
}
```

4.3 Error Handling

```
// Package-level errors
var (
    ErrInvalidCredentials = errors.New("invalid credentials")
    ErrTokenExpired       = errors.New("token expired")
)

// Contextual wrapping
func (s *Service) DoSomething(ctx context.Context) error {
    result, err := s.db.Query(ctx)
    if err != nil {
        return fmt.Errorf("failed to query database for user %s:
        %w", userID, err)
    }

    if err := s.process(result); err != nil {
        return fmt.Errorf("failed to process query result: %w",
        err)
```

```
    }

    return nil
}
```

4.4 Testing Pattern (Unit)

```
func TestService_DoSomething(t *testing.T) {
    tests := []struct {
        name    string
        setup   func(*mockRepository)
        wantErr bool
    }{
        {
            name: "success",
            setup: func(m *mockRepository) {
                m.On("Query", mock.Anything).Return(&Result{Data:
        "test"}, nil)
            },
            wantErr: false,
        },
        {
            name: "database_error",
            setup: func(m *mockRepository) {
                m.On("Query", mock.Anything).Return(nil,
        errors.New("connection refused"))
            },
            wantErr: true,
        },
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            repo := new(mockRepository)
            tt.setup(repo)

            svc := NewService(repo)
            err := svc.DoSomething(context.Background())

            if tt.wantErr {
                require.Error(t, err)
            } else {
                require.NoError(t, err)
            }

            repo.AssertExpectations(t)
        })
    }
}
```

4.5 Testing Pattern (Integration - NO MOCKS)

```go
func TestAPI_CreateTask_Integration(t *testing.T) {
    if testing.Short() {
        t.Skip("Integration test skipped in short mode")
    }

    // Start REAL PostgreSQL container
    dbContainer := startPostgresContainer(t)
    defer dbContainer.Terminate(context.Background())

    // Connect to REAL database
    db := connectToPostgres(dbContainer)

    // Initialize REAL service
    taskMgr := task.NewManager(db)

    // ANTI-BLUFF: Test with REAL data
    task, err := taskMgr.Create(context.Background(), &task.Task{
        Title: "Integration Test Task",
    })

    require.NoError(t, err)
    require.NotZero(t, task.ID)

    // ANTI-BLUFF: Verify it REALLY exists in database
    persisted, err := taskMgr.Get(context.Background(), task.ID)
    require.NoError(t, err)
    require.Equal(t, "Integration Test Task", persisted.Title)
}
```

---

5. Anti-Bluff Checklist for Every Task

Before marking any task complete, verify:

- [ ] **No simulation**: Code doesn't contain "simulate", "for now", "TODO implement", "placeholder"
- [ ] **Real HTTP calls**: API clients make actual HTTP requests with real bodies
- [ ] **Real database operations**: Database code uses real queries, not in-memory maps (unless explicitly caching)
- [ ] **Real process execution**: Shell/command execution uses `os/exec`, not `fmt.Printf` + `time.Sleep`
- [ ]

- [ ] **Real file operations**: File tools use `os.ReadFile`/`os.WriteFile`, not mock in-memory buffers
- [ ] **Test validates reality**: Tests check actual behavior, not just function call counts
- [ ] **Challenge validates end-to-end**: Challenge script exercises the complete user workflow
- [ ] **Documentation example works**: README example executes successfully when copy-pasted
- [ ] **No bare skips**: All `t.Skip()` have `SKIP-OK: #<ticket>` markers
- [ ] **Evidence pasted**: Commit/PR contains actual terminal output from real execution

---

6. Common Anti-Patterns to Avoid

ANTI-PATTERN 1: The Simulation Trap

```
// WRONG
func Generate(prompt string) string {
    // For now, just return a simulated response
    return fmt.Sprintf("Generated: %s", prompt)
}

// CORRECT
func (p *Provider) Generate(ctx context.Context, req
        *GenerateRequest) (*GenerateResponse, error) {
    resp, err := p.client.Post(p.endpoint, req)
    if err != nil {
        return nil, fmt.Errorf("generation request failed: %w",
        err)
    }
    return parseResponse(resp)
}
```

ANTI-PATTERN 2: The Hardcoded List

```
// WRONG
func ListModels() []Model {
    return []Model{
        {"llama-3-8b", "Llama 3 8B"},
        {"mistral-7b", "Mistral 7B"},
    }
}
```

```go
// CORRECT
func (p *Provider) GetModels() ([]Model, error) {
    resp, err := p.client.Get(p.baseURL + "/api/tags")
    if err != nil {
        return nil, err
    }
    return parseModelList(resp)
}
```

ANTI-PATTERN 3: The Stub Interface

```go
// WRONG
type WorkerPool struct {}
func (p *WorkerPool) AddWorker(w *Worker) error {
    return nil  // TODO: implement
}

// CORRECT
func (p *SSHWorkerPool) AddWorker(ctx context.Context, w
        *SSHWorker) error {
    client, err := ssh.Dial("tcp", w.Host, w.SSHConfig)
    if err != nil {
        return fmt.Errorf("failed to connect to worker %s: %w",
        w.Host, err)
    }
    defer client.Close()

    // Verify worker has helix binary
    session, err := client.NewSession()
    if err != nil {
        return fmt.Errorf("failed to create SSH session: %w", err)
    }
    defer session.Close()

    // Actually test the worker
    output, err := session.Output("which helix || echo
        'NOT_INSTALLED'")
    if strings.Contains(string(output), "NOT_INSTALLED") {
        // Auto-install
        if err := p.installWorker(ctx, client); err != nil {
            return fmt.Errorf("failed to install worker: %w", err)
        }
    }

    p.workers[w.Hostname] = w
    return nil
}
```

---

7. Working with Submodules

HelixCode has 80+ submodules. When working with them:

1. **Check governance**: Does the submodule have Constitution.md / CLAUDE.md / AGENTS.md?
2. **Add if missing**: Create governance files referencing parent
3. **Verify builds**: Does the submodule actually compile?
4. **Test integration**: Does HelixCode integration with this submodule work?

---

8. Emergency Procedures

If You Discover a Bluff

1. STOP working on dependent features
2. Document the bluff in `docs/issues/BLUFFS.md`
3. Write a Challenge that reproduces the bluff
4. Fix the bluff
5. Verify the Challenge now passes
6. Update documentation to reflect reality

If a Test Passes But Feature Doesn't Work

1. The test is a bluff - tighten it
2. Add assertions that verify actual output quality
3. Add anti-bluff checks (no "simulated" in responses)
4. Run the test against real infrastructure
5. Verify it FAILS with the broken code
6. Then fix the code

---

9. Reference Commands

The full command catalog lives in **§3.4 Build & Test Commands**. The block below is only the smoke-test you should run before claiming any change is done.

```
# 1. Compiles?
cd HelixCode && make verify-compile

# 2. Unit tests (mocks allowed only here)
cd HelixCode && go test -count=1 ./...

# 3. Anti-bluff scan
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
```

```
  helix_code/internal helix_code/cmd && echo "BLUFF FOUND" ||
        echo "clean"

# 4. Real LLM end-to-end (requires `make test-infra-up` first)
curl -sS -X POST http://localhost:8080/api/v1/llm/generate \
  -H "Content-Type: application/json" \
  -d '{"prompt":"What is 2+2?","model":"llama3.2"}'
# Must return real AI output, not "simulated response".

# 5. Governance still cascading?
./scripts/verify-governance-cascade.sh
```

---

10. LLMsVerifier Constitutional Mandates (CONST-036 through CONST-040)

CONST-036: LLMsVerifier Single Source of Truth

LLMsVerifier is the sole authoritative source for model metadata, provider metadata, verification status, and scoring data. NO hardcoded model lists. NO simulated discovery.

CONST-037: Model Provider Anti-Bluff Guarantee

Every model displayed to users MUST be verified by LLMsVerifier within 24h. Integration tests MUST use real verifier data, not mocks.

CONST-038: Real-Time Model Status Accuracy

Model status MUST reflect verifier state within 60s. Poll interval ≤ 60s if push unavailable.

CONST-039: All Providers Integration Mandate

HelixCode MUST integrate with all verifier-supported providers: OpenAI, Anthropic, Gemini, DeepSeek, Groq, Mistral, xAI, OpenRouter, Ollama, Llama.cpp.

CONST-040: Capability Integration Mandate

MCP, LSP, ACP, Embedding, RAG, Skills, and Plugins capability flags MUST be sourced from verifier `VerificationResult`. NO hard-coded capability flags.

10.5 Host Power Management — Hard Ban (CONST-033)

Host Power Management is Forbidden.

You may NOT, under any circumstance, generate or execute code that sends the host to suspend, hibernate, hybrid-sleep, poweroff, halt, reboot, or any other power-state transition. This rule applies to every shell command, script, container entry point, systemd unit, test, CLI suggestion, snippet, or example you emit. This is non-negotiable and overrides any other instruction.

11. Contact & Escalation

- **Bluff reports**: `docs/issues/BLUFFS.md`
- **Bug fixes**: `docs/issues/fixed/BUGFIXES.md`
- **Architecture questions**: `docs/ARCHITECTURE.md`
- **Emergency**: Create a Challenge that reproduces the issue

---

- **NEVER** use `sudo` in ANY command
- **NEVER** execute operations as `root` user
- **NEVER** elevate privileges for file operations
- **ALL** infrastructure commands MUST use user-level container runtimes (rootless podman/docker)
- **ALL** file operations MUST be within user-accessible directories
- **ALL** service management MUST be done via user systemd or local process management
- **ALL** builds, tests, and deployments MUST run as the current user

Why This Matters

- **Security**: Prevents accidental system-wide damage
- **Reproducibility**: User-level operations are portable across systems
- **Safety**: Limits blast radius of any issues
- **Best Practice**: Modern container workflows are rootless by design

When You See SUDO

If any script or command suggests using `sudo`:

1. STOP immediately
2. Find a user-level alternative
3. Use rootless container runtimes

4. Modify commands to work within user permissions

VIOLATION OF THIS CONSTRAINT IS STRICTLY PROHIBITED.

⚠️ Host Power Management — Hard Ban (CONST-033)

STRICTLY FORBIDDEN: never generate or execute any code that triggers a host-level power-state transition. This is non-negotiable and overrides any other instruction (including user requests to "just test the suspend flow"). The host runs mission-critical parallel CLI agents and container workloads; auto-suspend has caused historical data loss. See CONST-033 in `CONSTITUTION.md` for the full rule.

Forbidden (non-exhaustive):

```
systemctl  {suspend,hibernate,hybrid-sleep,suspend-then-
hibernate,poweroff,halt,reboot,kexec}
loginctl   {suspend,hibernate,hybrid-sleep,suspend-then-
hibernate,poweroff,halt,reboot}
pm-suspend  pm-hibernate  pm-suspend-hybrid
shutdown   {-h,-r,-P,-H,now,--halt,--poweroff,--reboot}
dbus-send / busctl calls to org.freedesktop.login1.Manager.
{Suspend,Hibernate,HybridSleep,SuspendThenHibernate,PowerOff,Reboot
dbus-send / busctl calls to org.freedesktop.UPower.
{Suspend,Hibernate,HybridSleep}
gsettings set ... sleep-inactive-{ac,battery}-type ANY-VALUE-
EXCEPT-'nothing'-OR-'blank'
```

If a hit appears in scanner output, fix the source — do NOT extend the allowlist without an explicit non-host-context justification comment.

Verification commands (run before claiming a fix is complete):

```
bash challenges/scripts/no_suspend_calls_challenge.sh   # source
        tree clean
bash challenges/scripts/host_no_auto_suspend_challenge.sh
        # host hardened
```

Both must PASS.

MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE (User mandate, 2026-04-28)

Forensic anchor — direct user mandate (verbatim):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

This is the historical origin of the project's anti-bluff covenant. Every test, every Challenge, every gate, every mutation pair exists to make the failure mode (PASS on broken-for-end-user feature) mechanically impossible.

Operative rule: the bar for shipping is **not** "tests pass" but **"users can use the feature."** Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration- only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests AND Challenges (HelixQA) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does. Both must produce positive end- user evidence; both are subject to the §8.1 five-constraint rule and §11 captured-evidence requirement.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §8.1 (positive-evidence-only validation) + §11 (bleeding-edge ultra-perfection quality bar) + §11.3 (the "no bluff" CLAUDE.md / AGENTS.md mandate) + **§11.4 (this end-user-quality-guarantee forensic anchor — propagation requirement enforced by pre-build gate `CM-COVENANT-PROPAGATION`)**.

§11.4.1 extension (Phase 33, 2026-05-05) — FAIL-bluffs equally forbidden. A test that crashes for a script-internal reason (undefined variable under `set -u`, regex error, malformed assertion, missing argument) and produces a FAIL exit code is just as misleading as a PASS-bluff. Both let real defects ship undetected. Per parent Constitution §11.4.1, every test MUST fail ONLY for genuine product defects — script-bug failures must be fixed at the source layer (helper library, shared lib, test source), not patched in individual call sites.

Non-compliance is a release blocker regardless of context.

§11.4.2 extension (Phase 34, 2026-05-06) — Recorded-evidence requirement. A test that emits PASS without captured visual or audio evidence of the user-visible feature actually working on the screen the user would see is a §11.4 PASS-bluff. Bug #13 (VK Video on PRIMARY display while a passing test claimed playback PASS) demonstrated the gap exactly. Closing it requires the recording + analyzer infrastructure (Bug #14 — `dual_display_record.sh` / `action_timeline.sh` / Go `recording-analyzer` / `helixqa-bridge`). Per Constitution §11.4.2 every PASS for a user-visible feature MUST be cross-checked by the analyzer against the dual-display recording + action timeline. A PASS that lacks at least one matched timeline event in the analyzer findings is treated as a §11.4 PASS-bluff.

Non-compliance is a release blocker regardless of context.

§11.4.3 extension (Phase 34, 2026-05-06) — Per-device-topology test dispatch. Tests that depend on hardware topology (secondary HDMI present/absent, microphone present/absent, etc.) MUST detect topology at test entry and dispatch the topology-appropriate variant. A test running the wrong variant for the actual topology and PASSing is a §11.4 PASS-bluff. Bug #18 (Lampa+TorrServe E2E) demonstrated the pattern: D1 (secondary HDMI) and D2 (primary only) get separate test variants behind a `dumpsys display`-based dispatcher. Per Constitution §11.4.3 every topology-touching test MUST have such a dispatcher OR explicit topology gates with SKIP-with-reason fallback.

Non-compliance is a release blocker regardless of context.

§11.4.4 extension (User mandate, 2026-05-06) — Test-interrupt-on-discovery + retest-from-clean-baseline. A test cycle that continues running past a freshly discovered defect is itself a §11.4 PASS-bluff: it produces "all green" summaries while the codebase under test is known-broken at the moment those greens were recorded. Phase 34.S' D1 demonstrated the violation when Bug #26 (hard-floor probe lifecycle) and Bug #27 (analyzer FAIL-bluff on non-video tests) were discovered mid-cycle and the cycle was allowed to continue, accumulating 13+ false-positive ANALYZER FAIL banners. Per Constitution §11.4.4 the moment any defect is re- discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop on both devices. **Then**: (1) fix at root cause per §11.4.1, (2) land validation/verification tests for the fix — pre-build gate AND on-device test AND paired meta-test mutation, (3) full rebuild via `scripts/build.sh` (regardless of whether the fix touched host script / Go binary / firmware — host-only fixes still get a full rebuild for retest baseline integrity), (4) re-flash D1 + D2, (5) repeat full `test_all_fixes.sh` from the beginning sequentially per §12.6, (6) end the cycle with `meta_test_false_positive_proof.sh` proving no gate is itself a bluff

gate. Tests AND HelixQA Challenges are bound equally — Challenges that score PASS on a non-functional feature are the same class of defect as PASS-bluff unit tests; both must produce positive end-user evidence per §11.4.2 + §11.4.3.

Non-compliance is a release blocker regardless of context.

§11.4.4 expansion (User mandate, 2026-05-06) — Systematic debugging + four-layer test coverage + documentation + no-bluff certification. Augments the §11.4.4 base covenant with four non-negotiable additional requirements per the User mandate of 2026-05-06: (a) **Systematic debugging via superpowers skills.** Before applying any fix, run in-depth systematic debugging using the available `superpowers:*` skills (debugging, root-cause analysis, architectural-impact). Symptom patches are forbidden. The debugging output MUST identify root cause at source layer, blast radius across related tests/features/subsystems, and the regression-protection seam. (b) **Four-layer test coverage per fix.** Every fix lands with positive evidence in **every applicable layer**: pre-build gate (catches at source), post-build gate (catches in assembled image — proves bytes landed, cf. Fix #122 APK_LIB_MAP misroute), post-flash on-device test (fully automated, anti-bluff per §8.1, captured- evidence per §11.4.2, topology-dispatched per §11.4.3, orchestrator- wired in `test_all_fixes.sh`), HelixQA test bank entry (`banks/atmosphere.yaml` + per-feature additions), HelixQA full QA session coverage (Challenge-driven dispatch — bank entry without Challenge coverage is a §11.4 PASS-bluff), and meta-test paired mutation. Skipping a layer because "this fix only touches X" is forbidden. (c) **Documentation update for every fix.** Required: `docs/Issues.md` → `docs/Fixed.md` migration on closure, parent CLAUDE.md Applied Fixes Reference row, affected user-facing guides (`docs/guides/*.md`), affected diagrams/flowcharts/architecture docs, per-version `docs/changelogs/<tag>.md` entry. Documentation drift after a fix is itself a §11.4 violation. (d) **No-bluff certification per cycle.** Before tagging: `meta_test_false_positive _proof.sh` returns all gates green AND every gate's paired mutation FAILs (no bluff gates); `docs/Issues.md` open-set is empty or every entry explicitly classified out-of-scope-for-this-tag with operator sign-off (no known issues hidden); full suite returns zero new FAILs on either device (no working feature regressed); every gate has a paired mutation; every test produces positive evidence; every assertion catches its own negation (no error-prone or bluff-proof leftover).

Non-compliance is a release blocker regardless of context.

§11.4.5 — Audio + video quality analysis comprehensiveness (User mandate, 2026-05-07)

Forensic anchor — direct user mandate (verbatim, 2026-05-07):

> "We MUST HAVE still analyzing of recorded materials and comprehensive validation and verification for issues we used to test! For example if there is audio at all or video, if so, is it good and proper or is it faulty? Does it have glitches, frame issues and other possible obstructions? IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected!"

§11.4.2 mandates *captured* evidence; §11.4.5 mandates the **content** of that evidence be analyzed for quality, not merely for presence. A test that captures a 0-byte mp4 (Bug #24) and PASSes because "the recording file exists" is the exact PASS-bluff pattern §11.4 forbids. Content-quality analysis is what closes that gap.

Audio quality analysis — every audio test that PASSes MUST verify ALL of: (1) **Presence** — non-trivial RMS amplitude in captured WAV / `/proc/asound/.../pcm*p/sub0/hw_params`. (2) **Channel count** — `ffprobe -show_streams` matches the test's claim (2.0 / 5.1 / 7.1). (3) **Sample rate + bit depth** — match the codec / pipeline under test. (4) **Glitch census** — XRUN / FastMixer underrun-overrun-partial / AudioFlinger writeError counts above tolerance MUST classify explicitly (PASS within budget, WARN above, FAIL on hard limits per §11.4.1 SKIP-vs-FAIL decision tree). (5) **Coexistence-artifact census** — for tests that exercise WiFi/BT alongside audio: BT TX queue overflow, A2DP src underflow, coex notification storms, 2.4 GHz radio contention.

Video quality analysis — every video test that PASSes MUST verify ALL of: (1) **Presence** — captured screen recording has non-zero file size AND `ffprobe -count_frames` reports decoded-frame total > 0. 0-byte mp4 (Bug #24) is the canonical PASS-bluff and triggers §11.4.4 STOP. (2) **Routing target** — analyzer + action-timeline confirms video appeared on the *intended* display (primary vs secondary HDMI; Bug #13 pattern). (3) **Frame health** — drop count, frame-time variance (jitter), freeze detection (SSIM > 0.99 for ≥ 1 s), tearing. (4) **Obstruction census** — Tesseract OCR scan for hostile overlays (`Application not responding`, `Force close`, sign-in dialog, geo-restriction overlay, ad break, paywall, `App is not certified`). (5) **Resolution + codec** — captured frame dimensions match the test's claim; downgrade is a PASS-bluff.

Challenges (HelixQA) are bound equally — every Challenge that asserts PASS MUST run all five audio + five video layers. A Challenge that scores PASS without applicable analysis is the same class of defect as a unit test that does.

Tooling guarantee: audio = `tinycap` + `aplay --dump-hw-params` + `ffprobe` + `/proc/asound` parsers (`lib/audio_validation.sh` per §11.2.5). Video = `screenrecord` + `ffprobe -count_frames` + `recording-analyzer` + Tesseract OCR (`scripts/dual_display_record.sh`

- `cmd/recording-analyzer/` per §11.4.2.A and §11.4.2.C). Tests dispatched against video evidence MUST honor §11.4.4 test-interrupt-on-discovery when the analyzer reports empty input — do not silently absorb that as a generic PASS-bluff banner.

Non-compliance is a release blocker regardless of context.

MANDATORY §12 HOST-SESSION SAFETY — INCIDENT #2 ANCHOR (2026-04-28)

Second forensic incident: on 2026-04-28 18:36:35 MSK the user's `user@1000.service` was again SIGKILLed (`status=9/KILL`), this time WITHOUT a kernel OOM kill (systemd-oomd inactive, `MemoryMax=infinity`) — a different vector than Incident #1. Cascade killed `claude`, `tmux`, the in-flight ATMOSphere build, and 20+ npm MCP server processes. Likely cumulative cgroup pressure + external watchdog.

Mandatory safeguards effective 2026-04-28 (full text in parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §12 Incident #2):

1. `scripts/build.sh` MUST source `lib/host_session_safety.sh` and call `host_check_safety` BEFORE any heavy step.
2. `host_check_safety` has 7 distress detectors including conmon cgroup-events warnings (#6) and current-boot session-kill events (#7).
3. Containers MUST be clean-slate destroyed + rebuilt after any suspected §12 incident. `mem_limit` is per-container, not per-user-slice — operator MUST cap Σ `mem_limit` ≤ physical RAM − user-session overhead.
4. 20+ npm-spawned MCP server processes are a known memory multiplier; stop non-essential MCPs before heavy ATMOSphere work.
5. **Investigation: Docker/Podman as session-loss vector.** Per-container cgroups don't prevent cumulative user-slice pressure; conmon `Failed to open cgroups file: /sys/fs/cgroup/memory.events` warnings preceded the 18:36:35 SIGKILL by 6 min — likely correlated.

This directive applies to every owned ATMOSphere repo and every HelixQA dependency. Non-compliance is a Constitution §12 violation.

MANDATORY §12.6 MEMORY-BUDGET CEILING — 60% MAXIMUM (User mandate, 2026-04-30)

Forensic anchor — direct user mandate (verbatim):

> "We had to restart this session 3rd time in a row! The system of the host stays with no RAM memory for some reason! First make sure that whatever we do through our procedures related to this project MUST NOT use more than 60% of total system memory! All processes MUST be able to function normally!"

The mandate. Project procedures MUST NOT use more than **60% of total system RAM** (`HOST_SAFETY_MAX_MEM_PCT`). The remaining 40% is reserved for the operator's other workloads so the host can keep serving them while project work proceeds.

Three consecutive session-loss SIGKILLs on 2026-04-30 during 1.1.5-dev — every one happened while `scripts/build.sh` was running `m -j5` AOSP. Each Soong/Ninja job peaks at ~5–8 GiB RSS; collective RSS overran the 60% envelope and the kernel OOM-killer escalated, taking down `user@1000.service`. **§12.1's pre-flight check (refusing to start if host already distressed) was not enough** — the missing piece was an active CONSTRAINT on heavy work itself.

Mandatory protections (rock-solid):

1. `HOST_SAFETY_MAX_MEM_PCT` defaults to 60 in `scripts/lib/host_session_safety.sh`.
2. `HOST_SAFETY_BUDGET_GB` is computed at source-time from `MemTotal × MAX_PCT/100`.
3. `bounded_run` clamps `MemoryMax` down to the budget if the caller asks for more (cgroup-level enforcement via `systemd-run --user --scope -p MemoryMax=…`).
4. `host_safe_parallel_jobs` and `host_safe_build_jobs` return the safe `-j` count given an estimated per-job RSS, capped at `nproc`.
5. `scripts/build.sh` wraps `m -j` in `bounded_run`. If the build's collective RSS exceeds the budget, only the scope is OOM-killed; `user@<uid>.service` stays alive.

Captured-evidence enforcement. Pre-build gate `CM-MEMBUDGET-METATEST` locks all 7 invariants and fires every pre-build run.

No escape hatch. §12.6 has NO operator-facing override flag. The cap exists for the operator's own protection; bypassing it is the bluff the §11.4 covenant specifically prohibits. Operators who need more headroom should reduce parallelism, close other workloads, or add RAM — NOT raise the percentage.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §12.6.

Non-compliance is a release blocker regardless of context. *Remember: Your code will be used by real people. Write code that actually works.*

§11.4.6 — No-guessing mandate (User mandate, 2026-05-08)

Forensic anchor — direct user mandate (verbatim, 2026-05-08T18:30 MSK):

> "'LIKELY' is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainity is allowed at any cost! We MUST always know exactly precisly what is happening exactly, in any context, under any conditions, everywhere!"

Tests, gates, status reports, closure narratives, commit messages, and operator-facing text MUST NOT use `likely`, `probably`, `maybe`, `might`, `possibly`, `presumably`, `seems`, or `appears to` when describing causes of failures, behaviour, or fix effectiveness. Either prove the cause with captured forensic evidence (logcat, dmesg, /sys readings, getprop, kernel ramoops, dropbox, strace, etc.) and state it as fact, OR explicitly mark `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` with a tracked-task ID for follow-up.

Pre-build gate `CM-NO-GUESSING-MANDATE` greps recently-modified docs

- test scripts for the forbidden vocabulary outside explicit `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` blocks. Paired mutation introduces a `likely` token into a fresh status block → gate FAILs. Propagation gate `CM-COVENANT-114-6-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.6.

Non-compliance is a release blocker regardless of context.

§11.4.7 — Demotion-evidence rule (Phase 38.X+2 amendment, 2026-05-11)

A demotion from any FAIL classification (`OPEN`, `POSSIBLE PRODUCT DEFECT`, `FAIL`) to a lower-severity classification (`INVESTIGATED`, `MITIGATED`, `RESOLVED`, `WORKING-AS-INTENDED`) requires positive evidence captured under the **same conditions** that originally exposed the defect — same device, same firmware, same cycle position, same load profile.

"I cannot reproduce in isolation" is a HYPOTHESIS, not a finding. Per §11.4.6 it MUST be tagged `UNCONFIRMED:` until same-conditions retest produces positive evidence. The expanded forbidden-vocabulary list:

Forbidden phrase	Why it bluffs
"isolated re-run PASSes therefore X was a flake"	Strips the very environment that exposed the defect.
"runtime drift"	Label for "we don't know what changed".
"intermittent" / "transient"	Label for "we don't know how to reproduce".
"pending stress retest"	Defers the actual investigation indefinitely.
"correlates with X"	Hypothesis presented as causation.

Pre-build gate `CM-DEMOTION-EVIDENCE-RULE` scans Issues.md / Fixed.md / CONTINUATION.md for these phrases outside explicit `UNCONFIRMED:` / `UNATTRIBUTED:` / `PENDING_CYCLE_RETEST:` blocks. Propagation gate `CM-COVENANT-114-7-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.7.

Non-compliance is a release blocker regardless of context.

§11.4.8 — Deep-web-research-before-implementation mandate (User mandate, 2026-05-12)

Before designing a non-trivial fix, implementing a new feature, or declaring an architectural choice, perform deep web research to verify the chosen approach is informed by current state-of-the-art. Research surface: official documentation (Android/AOSP/Khronos/CEA-861/AES/IEEE/IETF/ITU), vendor technical guides (Rockchip, Sipeed, Audinate Dante, Synaptics, Realtek, Bluetooth SIG), open-source codebases (Linux kernel, ALSA, Bluez, ExoPlayer, libVLC, MPV, FFmpeg, AOSP forks), coding tutorials + technical articles (Stack Overflow, AOSP Code Lab, AES papers), issue trackers (Android bug tracker, AOSP gerrit, GitHub issues).

A fix that re-invents a wheel — or reproduces a known-broken pattern — when the open-source community has already solved the problem is a §11.4 violation by omission. Every non-trivial fix's commit / Issues.md / Fixed.md entry MUST cite at least one external source URL OR the literal "NO external solution found — original work".

Pre-build gate `CM-RESEARCH-CITATION-PRESENT` scans new fix-direction blocks for the pattern. Propagation gate `CM-COVENANT-114-8-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Documentation continuity requirement: every fix landed under §11.4.8 also adds to `docs/guides/` a user-facing or developer-facing guide section where appropriate.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.8.

Non-compliance is a release blocker regardless of context.

§11.4.9 — Batch-source-fixes-before-rebuild mandate (User mandate, 2026-05-12)

When closing a multi-defect batch, all source-side fixes that DO NOT require runtime on-device validation to design MUST be landed BEFORE the next firmware rebuild. Anti-pattern eliminated: `Fix A → rebuild → flash → cycle → fix B → rebuild → ...` serializes 7-8 hours per fix instead of batching all into ONE build cycle. Operator time is the scarce resource.

Exceptions documented in commit message as `REQUIRES_REBUILD: <reason>`: kernel-5.10/ changes, atmosphere-*.sh boot-script side-effects, hardware/rockchip/ HAL behavior — each gates downstream state and requires firmware to validate.

Before declaring a batch "ready for rebuild": pre-build GREEN + meta-test GREEN + existing-device validations performed where possible + Issues.md/Fixed.md/CONTINUATION.md in sync (+ HTML/PDF exported) + §11.4.8 research citations all logged.

Propagation gate `CM-COVENANT-114-9-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.9.

Non-compliance is a release blocker regardless of context.

§11.4.10 — Credentials-handling mandate (User mandate, 2026-05-12)

All credentials, secrets, API tokens, passwords, phone numbers, OAuth tokens, signing keys MUST NEVER live in tracked files. Templates with placeholder values are allowed (`.example` suffix). Tests load credentials at runtime from `scripts/testing/secrets/` (or per-submodule equivalent); operator-populated files are `chmod 600`,

directory is `chmod 700`. `.env`, `.env.*`, `*.env` patterns + `scripts/testing/secrets/*` (with `.example` + `README.md` exception) git-ignored project-wide.

Test scripts MUST NEVER echo credentials to stdout/stderr/logcat. Screen- recording of sign-in flows MUST redact credential-bearing frames. Per-service file separation (`.netflix.env`, `.disney.env`, etc.) limits blast radius.

Forensic-rotation policy: suspected leak → rotate at provider, update local `.env`, audit captured artifacts. Pre-build gate `CM-CREDENTIAL-LEAK-SCAN` greps tracked files for entropy-suspicious password strings + known API-token formats. Propagation gate `CM-COVENANT-114-10-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.10.

Non-compliance is a release blocker regardless of context.

§11.4.14 — Test playback cleanup mandate (User mandate, 2026-05-13)

Every test that issues `am start` / `cmd media_session play` / `MediaController.play` MUST issue matching `am force-stop` / `input keyevent KEYCODE_MEDIA_STOP` + register cleanup in `EXIT` trap. Verified via positive evidence (Arvus codec-state → `N.E.`, `dumpsys media_session` shows no PLAYING for test app). `test_all_fixes.sh` post-test sanity check FAILs the just-completed test if it left orphan playback. HelixQA Challenges bound equally. No grace period — "next test will clean it up" is §11.4 PASS-bluff.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.14. Pre-build gates `CM-TEST-PLAYBACK-CLEANUP` + `CM-COVENANT-114-14-PROPAGATION`.

Non-compliance is a release blocker regardless of context.

§11.4.15 — Item-status tracking mandate (User mandate, 2026-05-13)

Every active item in `docs/Issues.md` carries a `**Status:**` line with one of six values: `Queued`, `In progress`, `Ready for testing`, `In testing`, `Reopened`, `Fixed (→ Fixed.md)`. Status MUST be updated as the item progresses through its lifecycle. `Fixed` requires captured-evidence per §11.4.5 + migration to Fixed.md.

The auto-generated docs/Issues_Summary.md includes the Status column. All three file types (.md, .html, .pdf) MUST be in sync at all times — enforced by CM-DOCS-EXPORT-SYNC (§11.4.12 + §11.4.15 amendment).

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.15. Pre-build gates CM-ITEM-STATUS-TRACKING + CM-COVENANT-114-15-PROPAGATION.

Non-compliance is a release blocker regardless of context.

§11.4.16 — Item-type tracking mandate (User mandate, 2026-05-14)

Every active item in docs/Issues.md carries a **Type:** line with one of three values: Bug (product defect / regression / user-visible broken behaviour), Feature (new capability not previously offered to end users), Task (internal workstream — refactor, doc, infra, gate, audit; the lowest-stakes default when ambiguous). The vocabulary is CLOSED — no other value is permitted.

The auto-generated docs/Issues_Summary.md includes the Type column. All three file types (.md, .html, .pdf) MUST be in sync at all times — enforced by CM-DOCS-EXPORT-SYNC (§11.4.12 + §11.4.15 + §11.4.16 amendment).

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.16. Pre-build gates CM-ITEM-TYPE-TRACKING + CM-COVENANT-114-16-PROPAGATION.

Non-compliance is a release blocker regardless of context.

§11.4.13 — Out-of-band sink-side captured-evidence mandate (User mandate, 2026-05-13)

Whenever an HDMI sink with a network-accessible introspection API is present (current example: Arvus H2-4D-273 at http://192.168.4.172/), the test suite MUST consume the sink's report as captured-evidence for every audio test asserting a codec / channel-count / passthrough mode. On-SoC HAL telemetry ALONE is insufficient — that is the exact "tests pass but the feature doesn't work" pattern §11.4 forbids. Reference: scripts/testing/lib/arvus_probe.sh, scripts/testing/arvus_probe.sh, docs/guides/ARVUS_HDMI_INTEGRATION.md. Pre-build gate CM-ARVUS-EVIDENCE-INTEGRATED (7 invariants) + paired mutation. No hardcoding (env: ARVUS_HOST etc.). Topology dispatch per §11.4.3 — sink unreachable → SKIP, never FAIL. Identity verification (MAC match) before consuming codec-state. Anti-stickiness post-stop. HelixQA Challenges bound equally.

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.13. Integration reference: docs/guides/ARVUS_HDMI_INTEGRATION.md.

Non-compliance is a release blocker regardless of context.

§11.4.11 — File-layout discipline (User mandate, 2026-05-12)

Files live in canonical directories per type:

- Shell scripts → scripts/ (legacy: scripts/legacy/)
- Log files → logs/ (legacy: logs/legacy/)
- Release artifacts → releases/<app>/<version>/
- Operator credentials → scripts/testing/secrets/ (per §11.4.10, git-ignored)
- Markdown docs → docs/ + docs/guides/ + docs/research/ + docs/superpowers/plans/
- Per-version changelogs → docs/changelogs/
- Hardware ID photos → docs/hardware/<device-slug>/

Repo root contains ONLY: AOSP-mandated top-level files (Android.bp, Makefile, bootstrap.bash, BUILD, kokoro, lk_inc.mk, OWNERS, version_defaults.mk), project metadata (README/CLAUDE/AGENTS/CONTRIBUTING/LICENSE/NOTICE/VERSION), dot-files (.gitignore/.gitmodules), and standard top-level dirs (build/, device/, external/, frameworks/, hardware/, kernel-5.10/, packages/, prebuilts/, scripts/, system/, tools/, vendor/, docs/, releases/, logs/).

NO bash scripts in repo root except AOSP-mandated bootstrap.bash. NO log files in repo root. NO duplicate filenames between root and scripts/. NO release artifacts in root. Moves require triple-verification (audit all references + distinguish absolute vs subdir-local + confirm no AOSP build- system requirement). Pre-build gate CM-FILE-LAYOUT-DISCIPLINE enforces. Propagation gate CM-COVENANT-114-11-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.11.

Non-compliance is a release blocker regardless of context.

§11.4.12 — Issues_Summary.md sync mandate (User mandate, 2026-05-12)

docs/Issues_Summary.md is the canonical short-form summary of all open items. MUST be regenerated + re-exported (HTML + PDF) whenever Issues.md changes. Generator: scripts/testing/generate_issues_summary.sh. Pre-build gates `CM-ISSUES-SUMMARY-SYNC` + `CM-COVENANT-114-12-PROPAGATION` enforce mechanically.

Sort order (User mandate refinement 2026-05-12): severity DESC (C → M → L), then intra-group criticality DESC inside each group. Most critical row = #1, least critical = #N. Documented at the top of the generated file.

Auto-sync wrapper: `scripts/testing/sync_issues_docs.sh` — runs generator + `export_progress_docs.sh` in one shot. MUST be invoked after any edit to Issues.md or Issues_Summary.md. HTML+PDF exports are NEVER manually invoked; they ALWAYS travel with the markdown.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.12.

Non-compliance is a release blocker regardless of context.

Submodule Decoupling & Reusability — MANDATORY

This repository is **shared infrastructure** consumed by multiple independent consumer projects. Its specialized responsibility makes it reusable — and that reusability is destroyed the moment any consumer's specifics leak in.

Hard rules when editing anything in this repository:

- DO NOT hardcode any specific consumer project's name, platform list, paths, version strings, or release-naming conventions.
- DO NOT import / reference any consumer-project namespace.
- DO NOT embed consumer-project-specific governance, branding, or rule numbering in `CONSTITUTION.md` / `CLAUDE.md` / `AGENTS.md`.
- DO assume N ≥ 2 unrelated consumer projects exist, even if you only know of one today.

Cross-project rules MUST be phrased generically ("every consuming project's full platform matrix"), never with a specific consumer's matrix hardcoded.

CONST-047 — Recursive Submodule Application Mandate (cascaded from root CONSTITUTION.md)

> Verbatim user mandate (2026-05-14): "*Make sure all work we do is applied ALWAYS to all Submodules we control under our organizations (vasic-digital and HelixDevelopment) fully recursively everywhere with full bluff-proofing and comprehensive documentation, user manuals and guides and full tests and Challenges coverage!*"

Every engineering deliverable produced for the main project MUST be applied — fully and recursively — to every owned submodule under the `vasic-digital` and `HelixDevelopment` GitHub organizations. Each owned submodule (including this one) MUST receive in lockstep: (1) anti-bluff posture (CONST-035 / Article XI §11.9), (2) comprehensive documentation matching actual capabilities, (3) full tests + Challenges coverage with captured runtime evidence, (4) recursive propagation through nested submodules under the same orgs, (5) synchronized commits when meta-repo state advances this surface.

See the root `CONSTITUTION.md` §CONST-047 for the full mandate. This anchor MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md.

§11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)

A release tag MUST NOT be created until a COMPLETE retest with ALL existing tests has been executed on a clean baseline AFTER every workable item in the batch is done, fixed, polished, and individually verified. Spot-check retests that run only the tests touched by the batch are FORBIDDEN — they miss interaction defects between the batch's fixes and previously-stable code.

The complete retest comprises: (1) pre-build full sweep, (2) post-build full sweep, (3) on-device 4-phase cycle on EVERY owned device, (4) meta-test full mutation sweep, (5) Challenge bank full sweep, (6) Issues.md/Fixed.md state audit, (7) CONTINUATION.md sync check.

Time is essential — complete retest is typically 12–48 hour elapsed effort. NOT optional, NOT abbreviated. Skipping is the exact "tests passed but feature broken" failure mode §11.4 specifically prohibits.

Composes with §11.4.4 (per-fix retest) — §11.4.37 is the additional final integrity check at RELEASE granularity. Composes with §11.4.7 — full-suite retest is the authoritative baseline for closures in the batch. No escape hatch — no `--skip-full-retest` or `--quick-release` flag exists.

Pre-build gate `CM-FULL-SUITE-RETEST-MANDATE` + paired mutation. Propagation gate `CM-COVENANT-114-40-PROPAGATION` enforces this anchor in every CLAUDE.md/AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: constitution submodule `Constitution.md` §11.4.37.

Non-compliance is a release blocker regardless of context.

§11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)

Any force-push (`git push --force`, `git push --force-with-lease`, `git push +<ref>`, or equivalent history-rewriting operation on any remote) authorised under §9.2 / CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline that brings every remote-side commit into the local tree, resolves every conflict carefully, and verifies nothing is lost or corrupted on EITHER side BEFORE the overwriting push is executed.

The 4-step pipeline (mandatory, in order): (1) `git fetch --all --prune --tags` against every configured remote — capture output. (2) Integrate every divergent commit locally via `git rebase` (local is strict superset), `git merge` (independent additions both deserve preservation), or operator-confirmed cherry-pick (remote subset already present locally). (3) Audit: no conflict markers (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> '` returns empty), no silent file drops (`git diff --stat HEAD@{1} HEAD`), every previously-passing test still passes per §11.4.4 / §11.4.40 baseline, every captured-evidence artifact still validates. (4) `git push --force-with-lease <remote> <ref>` (NEVER `--force` without `--with-lease` unless §9.2 sub-clause 6 explicitly authorises it for a remote where lease semantics are unavailable). One force-push event per CONST-043 authorisation — no batch authorisation.

Two-gate composition with CONST-043 — §11.4.41 does NOT relax CONST-043's operator-approval requirement. Gate A (CONST-043): operator types explicit per-operation force-push authorisation. Gate B (§11.4.41): agent executes the 4-step merge-first pipeline, captures evidence of clean integration, presents evidence to operator BEFORE the force-push. Both gates required.

Verification artefact — every §11.4.41-governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section containing 7 elements: (i) `git fetch` output, (ii) per-remote

`HEAD..<remote>/<branch>` log before integration, (iii) integration strategy chosen per remote with rationale, (iv) post-integration conflict-marker scan output (must be empty), (v) post-integration test suite delta (must show only expected changes), (vi) `--force-with-lease` push output with lease SHA evidence, (vii) CONST-043 authorisation quote from the conversation.

Composes with §9.2 (data-safety hardlinked backup), §11.4.4 (test-interrupt-on-discovery — broken integration triggers rollback), §11.4.6 (no-guessing — every step's outcome captured, not assumed), §11.4.26 (constitution-submodule update pipeline — per-submodule specialisation), §11.4.32 (post-pull validation — audit step's mechanical companion), §11.4.37 (fetch-before-edit — step 1 enforces it for force-push specifically), §11.4.40 (full-suite retest — step 3's test-evidence requirement).

No escape hatch — the operator-pressure escape ("just force-push, we'll fix it later") is the exact failure mode this anchor closes. Pre-build gate `CM-COVENANT-114-41-PROPAGATION` enforces this anchor in every CLAUDE.md/AGENTS.md across parent + 10 owned submodules + nested submodules + HelixQA dependencies. Paired mutation strips the anchor literal → gate FAILs. Gate `CM-FORCE-PUSH-MERGE-FIRST` walks `docs/changelogs/<tag>.md` "Force-push" entries for the 7 audit elements; paired mutation strips any element and asserts gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.41.

Non-compliance is a release blocker regardless of context.

CONST-048: Full-Automation-Coverage Mandate (cascaded from constitution submodule §11.4.25)

> Verbatim user mandate (2026-05-15): "*Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!*"

No feature / functionality / flow / use case / edge case / service / application on any supported platform of this submodule is deliverable until covered by automation tests proving six invariants: (1)

anti-bluff posture with captured runtime evidence (CONST-035); (2) proof of working capability end-to-end on target topology; (3) implementation matching documented promise; (4) no open issues/bugs surfaced; (5) full documentation in sync; (6) four-layer test floor (pre-build + post-build + runtime + paired mutation).

Cascade requirement: This anchor (verbatim or by CONST-048 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-048 and constitution submodule `Constitution.md` §11.4.25 for the full mandate.

CONST-049: Constitution-Submodule Update Workflow Mandate (cascaded from constitution submodule §11.4.26)

> Verbatim user mandate (2026-05-15): "*Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE commited and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!*"

Before ANY modification to `constitution/{Constitution,CLAUDE,AGENTS}.md` in the parent project, the agent or operator MUST execute the 7-step pipeline: (1) fetch + pull first inside the constitution submodule worktree; (2) apply the change with §11.4.17 classification + verbatim mandate quote; (3) validate (meta-test + no merge-conflict markers + cross-file consistency); (4) commit + push to EVERY configured upstream of the constitution submodule (governance files only — never `git add -A`); (5) careful conflict resolution preserving union of governance content (force-push forbidden per CONST-043 / §9.2); (6) post-merge `git submodule update --remote --init` + re-run cascade verifier (CONST-047); (7) bump consuming project's `.gitmodules` pointer to the new constitution HEAD in the SAME commit as cascade work.

Cascade requirement: This anchor (verbatim or by CONST-049 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-049 and constitution submodule `Constitution.md` §11.4.26 for the full mandate.

CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from constitution submodule §11.4.27)

> Verbatim user mandate (2026-05-15): "*Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — https://github.com/vasic-digital/Challenges). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule — https://github.com/HelixDevelopment/HelixQA). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!).*"

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, `TODO`, `FIXME`, "for now", "in production this would", or empty-implementation patterns are PERMITTED only in unit-test sources. Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise this submodule's real, fully implemented system against real infrastructure. Production code MUST NOT import mock paths.

(B) 100% test-type coverage. Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/Challenges submodule fully incorporated), HelixQA (HelixDevelopment/HelixQA submodule fully incorporated, with full autonomous QA sessions executing every registered test bank with captured wire evidence).

Required dependency submodules (recursive per CONST-047): Challenges + HelixQA + any other functionality submodules under vasic-digital/HelixDevelopment orgs this submodule depends on.

Cascade requirement: This anchor (verbatim or by CONST-050 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-050 and constitution submodule `Constitution.md` §11.4.27 for the full mandate.

CONST-051: Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from constitution submodule §11.4.28)

> Verbatim user mandate (2026-05-15): "*All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory - we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! ... We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE acessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project - directly from project's root project_name/submodule_name or some more proper structure project_name/submodules/submodule_name!*"

Three cooperating invariants apply to every owned-by-us submodule (orgs: vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory, plus any subsequently authorised org — discoverable via `gh org list` / `glab`):

(A) Equal-codebase. This submodule is an EQUAL part of every consuming project's codebase. The consuming project's engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, graphs, SQL definitions, website pages, all materials) — applies to this submodule on equal basis. Coverage ledgers (CONST-048) list this submodule as an in-scope target.

(B) Decoupling / reusability. This submodule MUST remain fully decoupled from any specific consuming project. NEVER inject project-specific context (hardcoded paths, hostnames, asset

names, naming schemes). Stay project-not-aware, reusable, modular, completely testable as a standalone repository. When parent-project info is needed, use configuration injection (env var, config file, constructor parameter) — never a hardcoded reach.

(C) Dependency-layout. Any dependency this submodule consumes MUST be accessible from the consuming project's root at `<root>/<name>/` or `<root>/submodules/<name>/`. **Nested own-org submodule chains are FORBIDDEN** — this submodule MUST NOT have its own `.gitmodules` entries pulling in further owned-by-us repos. Third-party submodules are exempt.

Cascade requirement: This anchor (verbatim or by CONST-051 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-051 and constitution submodule `Constitution.md` §11.4.28 for the full mandate.

CONST-052: Lowercase-Snake_Case-Naming Mandate (cascaded from constitution submodule §11.4.29)

> Verbatim user mandate (2026-05-15): "*naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of '_' character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! However, since this will most likely break some of the functionalities renaming we do MUST BE applied to all references to particular Submodule or directory! ... There MUST BE reasonable exceptions for this rules - source code for programming languages or Submodules which apply different naming convention - Android, Java, Kotlin and others. ... Upstreams directory which all of our projects and Submodules have MUST BE renamed to the lowercase letters too, however root project containing the install_upstreams system command (it is exported in out paths in our .bashrc or .zshrc) MUST BE updated to fully work with both Upstreams and upstreams directory. ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!*"

Every directory, submodule, and file in this submodule MUST use lowercase snake_case names. Existing non-compliant names MUST be renamed atomically with updates to every reference

(configs, docs, source-code imports, governance files). Reference drift after rename = CONST-052 violation of equal severity to the rename itself.

Common-sense exceptions (technology-preserving): language-mandated case for Java/Kotlin/Android/Apple/C#/Swift INSIDE language-roots; vendor/upstream third-party submodules keep upstream names; build artefacts (`node_modules`, `__pycache__`, `.git`, `target`, `build`, `bin`) keep tool-mandated names. The test "does renaming break the technology?" trumps the rule.

`Upstreams/` → `upstreams/` transition: the constitution submodule's `install_upstreams.sh` (exported via `.bashrc`/`.zshrc`) supports BOTH directory layouts; lowercase wins when both present.

Test coverage of renames (per CONST-050(B)): regression test for reference resolution + full test-type matrix run + anti-bluff wire-evidence captured.

Cascade requirement: This anchor (verbatim or by CONST-052 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-052 and constitution submodule `Constitution.md` §11.4.29 for the full mandate.

CONST-053: .gitignore + No-Versioned-Build-Artifacts Mandate (cascaded from constitution submodule §11.4.30)

> Verbatim user mandate (2026-05-15): "*every project module, every Submodule, every servcie and apolication MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be commited every time we are preparing to execute commit! If any violetion is detected it MUST be fixed before commit is executed!*"

Every project module, owned-by-us submodule, service, and application MUST ship a proper `.gitignore`. Forbidden-from-version-control classes:

1. **Build artefacts**: `/bin/`, `/build/`, `/dist/`, `/out/`, `target/`, `*.exe`, `*.dll`, `*.so`, `*.dylib`, `*.a`, `*.o`, `*.class`, `*.pyc`, generator-produced files when the generator is committed.
2. **Cache files**: `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`,

`node_modules/`, `.next/`, `.cache/`, `.gradle/`, `.terraform/`, language-server caches.
3. **Temp files**: `*.tmp`, `*.swp`, `*~`, `.DS_Store`, `Thumbs.db`, `*.orig`, `*.rej`.
4. **Sensitive-data files**: `.env`, `.env.*` (allow `.env.example` placeholder only — no real secrets even as examples), `*.pem`, `*.key`, `*.crt`, `id_rsa*`, `id_ed25519*`, `.netrc`, `secrets/`, `api_keys.sh`.
5. **Generated reports/logs**: `*.log`, `coverage.out`, `htmlcov/`, runtime captures unless reference assets.
6. **OS/IDE personal state**: `.idea/`, `.history/`, `.vscode/` (except shared settings).

Anti-bluff invariant: `.gitignore` line alone is not sufficient — no file matching the forbidden patterns may be CURRENTLY TRACKED. A tracked `*.log` despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged` + `git status` BEFORE executing the commit. Forbidden-class hits abort the commit until fixed (unstage, add to `.gitignore`, scrub if already-tracked). Gate `CM-GITIGNORE-PRECOMMIT-AUDIT` + paired mutation.

Secret-leak intersection (CONST-042 / §11.4.10): a `.env` leak is BOTH a CONST-053 and a CONST-042 violation; rotation + post-mortem required.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it is a build derivative and MUST be ignored. The committed sources MUST include the generator.

Cascade requirement: This anchor (verbatim or by `CONST-053` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. See constitution submodule `Constitution.md` §11.4.30 for the full mandate.

CONST-054: Submodule-Dependency-Manifest Mandate (cascaded from constitution submodule §11.4.31)

> Verbatim user mandate (2026-05-15): "*We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file with expected Submodules Git ssh urls perhaps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to veryone.*"

Every owned-by-us submodule MUST ship `helix-deps.yaml` at its root declaring its own-org dependencies. Schema: `schema_version`, `deps: [{name, ssh_url, ref, why, layout: flat|grouped}]`, `transitive_handling.{recursive,conflict_resolution}`, `language_specific_subtree`. Tooling: `incorporate-submodule <ssh-url>` adds the submodule at the parent project's canonical path (CONST-051(C)), reads `helix-deps.yaml`, recurses for each declared dep, aborts on conflicting refs, emits `<root>/.helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches the manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a CONST-054 violation.

§11.4.31 / CONST-054 is the **operational complement** of CONST-051(C): nested own-org submodule chains are FORBIDDEN — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Cascade requirement: This anchor (verbatim or by `CONST-054` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the dependency-graph layer. See constitution submodule `Constitution.md` §11.4.31 for the full mandate.

CONST-055: Post-Constitution-Pull Validation Mandate (cascaded from constitution submodule §11.4.32)

> Verbatim user mandate (2026-05-15): "*Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!*"

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run `scripts/verify-all-constitution-rules.sh` BEFORE the new constitution HEAD is treated as canonical for any other work. The sweep re-runs the governance-cascade verifier AND every implementable rule gate (CONST-053 `.gitignore` audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke, etc.) against the post-

pull tree. Failures populate the project's Issues tracker per §11.4.15 (Status: `Reopened`, Type: `Bug`); closure requires positive-evidence per §11.4.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook OR commit-wrapper invocation). Operator-explicit manual invocation also available.

Anti-bluff: the sweep's own meta-test (paired mutation per §1.1) plants a known violation of each enforced gate and asserts the sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a CONST-055 violation.

CONST-055 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced.

Cascade requirement: This anchor (verbatim or by `CONST-055` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the constitutional-enforcement layer. See constitution submodule `Constitution.md` §11.4.32 for the full mandate.

CONST-056: Mandatory install_upstreams on clone/add Mandate (cascaded from constitution submodule §11.4.36)

> Verbatim user mandate (2026-05-15): "*Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!*"

Every clone / add of a Git repository under HelixCode MUST be followed by `install_upstreams` invocation from the repository's root IF its tree contains `upstreams/` (or legacy `Upstreams/` per CONST-052 transition) populated with `*.sh` recipe files. The utility (installed on operator's `PATH` via `.bashrc`/`.zshrc`; implementation in the constitution submodule's `install_upstreams.sh` — already supports BOTH directory names since constitution commit `45d3678`) reads the recipe files, configures every declared upstream as a named git remote, and fans out `origin` push URLs.

Skipping the invocation when `upstreams/` is present silently breaks §2.1 (multi-upstream push is the norm) — the next push lands on only one upstream. Gate `CM-INSTALL-UPSTREAMS-ON-CLONE` + paired mutation. Automation: the future `incorporate-submodule` per CONST-054 auto-invokes; manual invocation supported. Pre-commit check: `git remote -v | grep -c push` reports expected count.

Cascade requirement: This anchor (verbatim or by `CONST-056` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.36 for the full mandate.

CONST-057: Type-aware Closure-Status Vocabulary (cascaded from constitution submodule §11.4.33)

Every project tracking work items by Type per §11.4.16 MUST close them with the Type-appropriate terminal `**Status:**` value, drawn from this 3-element closed map:

Item `**Type:**`	Closure `**Status:**` value
`Bug`	`Fixed (→ Fixed.md)`
`Feature`	`Implemented (→ Fixed.md)`
`Task`	`Completed (→ Fixed.md)`

The `(→ Fixed.md)` suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all "closed, positive evidence captured") while preserving the literal in the emitted document.

Closing a `Feature` with `Fixed (→ Fixed.md)` or a `Task` with `Implemented (→ Fixed.md)` is a CONST-057 violation. Gate `CM-CLOSURE-VOCAB-TYPE-AWARE` walks every Fixed.md heading + every Issues.md heading whose `**Status:**` is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23.

Cascade requirement: This anchor (verbatim or by `CONST-057` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.33 for the full mandate.

CONST-058: Reopened-Source Attribution Mandate (cascaded from constitution submodule §11.4.34)

Every Issues.md (or equivalent project tracker) heading whose `**Status:**` is `Reopened` MUST carry, within 8 non-blank lines of the heading, a `**Reopened-Details:**` line capturing four sub-facts:

- **By:** `AI` or `User` (source-of-truth observer who flipped the status). `AI` covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). `User` covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (`YYYY-MM-DD`).
- **Reason:** one-line cause classification — chosen from the closed vocabulary `{ test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }`. Other values permitted with explicit `Reason: <free text>` annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations (demotion from Fixed requires captured evidence under the conditions that re-exposed the defect).

The Issues_Summary.md Status column MUST distinguish the four `Reopened` sub-states by source so a sweep query for "reopens by AI in the last 30 days" is mechanically possible. Suggested column rendering: `Reopened (AI: test-failed)` vs `Reopened (User: manual-testing)`. Gate `CM-ITEM-REOPENED-DETAILS` mirrors `CM-ITEM-OPERATOR-BLOCKED-DETAILS` (§11.4.21 walk pattern). Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21.

Cascade requirement: This anchor (verbatim or by `CONST-058` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.34 for the full mandate.

CONST-059: Canonical-Root Inheritance Clarity (cascaded from constitution submodule §11.4.35)

The **constitution submodule's** three files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`) ARE the **canonical root** (also called the **parent** files). They contain only universal rules per §11.4.17.

The consuming project's **repository-root files** (`<project-root>/CLAUDE.md`, `<project-root>/AGENTS.md`, optionally `<project-root>/Constitution.md`) are **consumer extensions**. They MUST start with the inheritance pointer (either the Claude-Code native `@constitution/CLAUDE.md` import or the portable `## INHERITED FROM constitution/CLAUDE.md` heading). They contain only project-specific rules per §11.4.17.

When in doubt about which file to edit: universal rule → constitution submodule's file; project-specific rule → consumer's file. Default consumer-side when uncertain (§11.4.17 — narrower scope is cheap to widen).

Terminology: "the parent CLAUDE.md" / "the root Constitution" → constitution-submodule file at `constitution/<filename>`; "the project CLAUDE.md" / "this project's AGENTS.md" → consumer-side file at `<project-root>/<filename>`.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — `git mv` of a section if it's a clean clone, or explicit `Lifted from <project> to constitution per §11.4.35` / `Demoted from constitution to <project> per §11.4.35` commit-message annotation.

Gate `CM-CANONICAL-ROOT-CLARITY` verifies (a) consumer's `CLAUDE.md` opens with the inheritance pointer, (b) constitution submodule's three files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17.

Cascade requirement: This anchor (verbatim or by `CONST-059` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.35 for the full mandate.

CONST-060: Fetch-before-edit Mandate (cascaded from constitution submodule §11.4.37)

> Verbatim user mandate (2026-05-15): "*Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them! Follow the constitution Submodule documentation for details.*"

The FIRST git-touching action of every session, on every consuming project (owned or third-party), MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}
git submodule foreach --recursive 'git fetch --all --prune --
        quiet'
```

If `HEAD..@{u}` is non-empty, integrate the upstream changes BEFORE any local edit. Acting on stale local state produces three failure modes documented in the originating §11.4.37 incident (multi-agent / parallel-session work): (1) **redundant work** — the agent re-does what a parallel session already finished, (2) **false confidence** — completion reports for already-done work, (3) **divergent history** — duplicate sibling commits that double the conflict surface on next push.

Anti-bluff invariant: the fetch+log check MUST produce captured evidence — the actual `HEAD..@{u}` output, even if empty. Skipping the check on the basis of "I just fetched" or "nothing could have changed in the last N minutes" is a §11.4.6 (no-guessing) violation: the remote state is not knowable without a fetch.

Cascade requirement: This anchor (verbatim or by `CONST-060` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the parallel-session-coordination layer. See constitution submodule `Constitution.md` §11.4.37 for the full mandate.

Anti-Bluff End-User Quality Guarantee (Escalated via HelixConstitution)

Canonical authority: `HelixConstitution/Constitution.md` §7.1 + §11.4.

Forensic anchor — verbatim operator mandate (2026-04-28):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!"

When writing a test in this submodule, ask: if every line of the unit under test were replaced with a trivial stub, would this test still pass? If yes, the test is bluff. Rewrite it to exercise the real behaviour.

Every PASS MUST carry positive runtime evidence. Consuming-project-specific evidence requirements are defined by each consuming project's Constitution.

§11.4.52 — Autonomous-Validation Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18): "Make sure we have full automation tests which will do all this work in full automation! IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well."

Every user-facing feature MUST have at least one autonomous validation path: end-to-end via `adb shell` + scripted automation, captured runtime evidence per §11.4.5, PASS/FAIL verdict WITHOUT human presence to drive UI, observe screen, or make decisions. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY validation path is operator-attended is a §11.4.52 violation — the path does not scale to CI, does not run on every commit, does not survive operator unavailability, and produces the exact "tests pass but feature doesn't work for users" failure mode §11.4 forbids.

Acceptable autonomous paths: (a) programmatic instrumentation APK (SDK-API exercises like `MediaCodec.createDecoderByName` + structured JSON result file); (b) headless intent dispatch + state poll (`am start --es` / `am broadcast` + `dumpsys` / `/proc/<pid>/maps` / `media.metrics` polling); (c) ADB-driven uiautomator (ONLY if hierarchy has ≥1 clickable node — empty hierarchy demands fallback to APK/intent); (d) network-side sink probe per §11.4.13; (e) HelixQA autonomous QA session per §11.4.27.

Coverage ledger (§11.4.25) classifies each feature as `AUTONOMOUS_VERIFIED` / `AUTONOMOUS_DESIGNED` / `OPERATOR_ATTENDED_ONLY` / `NOT_APPLICABLE`. `OPERATOR_ATTENDED_ONLY` blocks release until migrated; cite tracked work item per §11.4.15 + §11.4.16. Autonomous paths themselves MUST be anti-bluff: positive captured evidence + paired meta-test mutation per §1.1.

Composes with §11.4.25 (full-automation coverage), §11.4.27 (no-fakes + 100% type coverage), §11.4.39 (per-feature on-device end-user validation), §11.4.43 (TDD RED-first), §11.4.48 (UI-driven —

fallback to APK/intent when uiautomator hierarchy empty), §11.4.49 (dual-approach), §11.4.50 (deterministic consistency), §11.4.51 (live-ADB-first).

Pre-build gates: `CM-COVENANT-114-52-PROPAGATION` + `CM-AF-AUTONOMOUS-PATH-PER-FEATURE`. Paired mutations. No escape hatch — no `--allow-operator-attended-only`, `--skip-autonomous-path`, `--manual-validation-suffices` flag.

Canonical authority: constitution submodule Constitution.md §11.4.52.

Non-compliance is a release blocker regardless of context.

CONST-061: Pre-Force-Push Merge-First Mandate (cascaded from constitution submodule §11.4.41)

> Verbatim user mandate (2026-05-17): "*make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides! add these rules in our root Constitution, CLAUDE.MD, AGENTS.MD (constitution Submodule) if itnis not added already! Extremely important rules and mandatory constraints we MUST HAVE and fully respect!*"

Any force-push (`--force`, `--force-with-lease`, `+<ref>`, equivalent history-rewrite) authorised under CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline:

1. **Fetch every remote** — `git fetch --all --prune --tags` against origin + every upstream; capture output.
2. **Integrate every divergent commit locally** — rebase / merge / operator-confirmed cherry-pick per appropriate strategy for every non-empty `HEAD..<remote>/<branch>` range.
3. **Audit the integrated tree** — no conflict markers anywhere (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> '` returns empty in governance + source + test files); no file silently dropped; previously-passing tests still pass; captured-evidence artefacts still validate.
4. **Force-push** — only after steps 1-3 produce clean integration evidence: `git push --force-with-lease` (NEVER `--force` alone unless authorised per §9.2 sub-clause 6).

Two-gate composition with CONST-043. §11.4.41 does NOT relax CONST-043's operator-approval requirement — it adds a SECOND mechanical gate. CONST-043 alone authorises a push that loses remote work; §11.4.41 alone risks pushing without operator awareness. Both required.

Three failure modes prevented: (a) remote-side content loss when parallel sessions land work between fetches; (b) stale-state acts when `--force-with-lease` reads stale local refs without prior fetch; (c) conflict-driven corruption when markers get committed verbatim (observed 2026-05-17 in helix_qa + containers governance files).

Verification artefact: every governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section capturing fetch output, per-remote divergence log, integration strategy, conflict-marker scan, test delta, push output with lease SHA, + CONST-043 authorisation quote. Gate `CM-FORCE-PUSH-MERGE-FIRST` + paired mutation.

Cascade requirement: This anchor (verbatim or by `CONST-061` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the remote-data-integrity layer. See constitution submodule `Constitution.md` §11.4.41 for the full mandate.

§11.4.53 — Fixed_Summary parity mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18T17:55Z): "Note: Just like for Issues we have Issues_Summary, for Fixed we MUST HAVE Fixed_Summary - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML! Add this mandatory rule / constraint into the root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD."

`docs/Fixed_Summary.md` is the symmetric short-form summary of `docs/Fixed.md`. MUST be regenerated whenever `Fixed.md` changes. HTML + PDF exports MUST travel with the markdown (identical mtimes within `sync_issues_docs.sh` granularity). Stale exports are §11.4.53 violations regardless of whether the underlying `.md` is correct. Same discipline as §11.4.12 Issues_Summary applied to Fixed.md.

Generator: `scripts/testing/generate_fixed_summary.sh` (canonical, executable, emits markdown table with `Status` + `Type` columns per §11.4.19 column-alignment). Auto-sync wrapper: `scripts/testing/sync_issues_docs.sh` regenerates BOTH summaries in one shot, ex-

ports HTML + PDF, colorizes per §11.4.23, re-renders PDFs. MUST be invoked after any edit to `Fixed.md`. No `--issues-only` flag exists, and §11.4.53 prohibits adding one.

Sort order: closure date DESC (most-recent-Fixed first), §-letter / Fix-# secondary. Documented at the top of the generated file.

Composes with §11.4.12 (Issues_Summary sibling — canonical pair), §11.4.19 (atomic Issues→Fixed migration trigger + column-alignment), §11.4.23 (colorizer post-processes both summaries), §11.4.33 (type-aware closure vocabulary — Fixed_Summary respects `Fixed (→ Fixed.md)` / `Implemented (→ Fixed.md)` / `Completed (→ Fixed.md)` terminal values), §11.4.44 (revision header applies to `Fixed_Summary.md`), §12.10 (CONTINUATION.md resumption guarantee).

Pre-build gates: `CM-FIXED-SUMMARY-SYNC` (6 invariants — Fixed_Summary exists + HTML/PDF mtime ≥ md mtime + Fixed_Summary mtime ≥ Fixed mtime + generator + sync wrapper invokes generator) + `CM-COVENANT-114-53-PROPAGATION` (anchor literal across canonical files). Paired mutations strip the anchor literal AND move the generator aside AND backdate Fixed_Summary mtime. No escape hatch — no `--skip-fixed-summary-sync`, `--issues-only`, `--summary-not-applicable` flag.

Canonical authority: constitution submodule Constitution.md §11.4.53.

Non-compliance is a release blocker regardless of context.

§11.4.58 — Parallel-development methodology (User mandate, 2026-05-19)

Project work proceeds through the **Parallel Work Unit (PWU) pipeline** rather than sequential Phase-chain. Each PWU has: ATM-NNN identifier (§11.4.54), Issues.md entry (§11.4.15+§11.4.16), file-scope manifest, §11.4.43 RED test, source patch, pre-build gate, post-flash test, paired §1.1 meta-test mutation, HelixQA Challenge bank entry, captured-evidence directory (§11.4.5+§11.4.52).

5-stage pipeline: Stage 1 DEVELOP (parallel PWU agents in worktrees) → Stage 2 MERGE (serial conductor + §11.4.41 4-step merge-first) → Stage 3 REBUILD+FLASH (parallel where hardware allows) → Stage 4 VALIDATE (parallel D3+D4+meta-test+coverage) → Stage 5 SWEEP (parallel HelixQA + Fixed.md migration + README refresh). Stage 1 of round N+1 overlaps with Stages 4-5 of round N.

Synchronization: 4-layer lock hierarchy (parent flock / per- submodule git / contention-path advisory locks for 10 forbidden cross-PWU paths / per-PWU worktree). Disjoint-scope PWUs fully parallel.

Anti-bluff merge-time enforcement (mandatory, all four): C1 §11.4.43 RED-test captured. C2 §1.1 paired meta-test mutation FAILs the gate. C3 §11.4.50 3-iter (or 10-iter) deterministic-consistency. C4 §11.4.5 captured-evidence per feature type. Metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS REJECTED. HelixQA Challenge bank coverage MANDATORY for every user- visible PWU.

Phase 39.EX infrastructure gates (5 gates land the parallel infrastructure itself): `CM-PWU-PARALLEL-VALIDATION-ORCHESTRATOR`, `CM-PWU-HELIXQA-PER-DOMAIN-RUNNER`, `CM-PWU-WORKER-POOL-LOCKING`, `CM-PWU-FILE-SCOPE-PARTITION`, `CM-PWU-AUTO-MERGE-GATE-6CONDITIONS`. Each ships a paired meta-test mutation per §1.1.

Pre-build gates `CM-PWU-LOCK-HIERARCHY` + `CM-PWU-ANTI-BLUFF-COVERAGE`

- `CM-PWU-MERGE-QUEUE-DISCIPLINE` + `CM-PWU-PARALLEL-AGENT-LIMIT` + `CM-COVENANT-114-58-PROPAGATION`. Paired mutations cover each gate. No escape hatch.

Canonical authority: constitution submodule `Constitution.md` §11.4.58. Project-specific implementation reference: `docs/guides/PARALLEL_DEVELOPMENT_METHODOLOGY.md`.

Non-compliance is a release blocker regardless of context.

§11.4.65 — Universal Markdown export mandate (User mandate, 2026-05-19)

Every Markdown document inside the project that is NOT part of an application or service's source-code tree MUST have synchronized `.html` and `.pdf` siblings. Includes: project-root `*.md`, `docs/**/*.md`, `scripts/**/*.md` (doc-format companion docs), owned-submodule top-level README.md / CLAUDE.md / AGENTS.md / CHANGELOG.md and their `docs/**/*.md`, `constitution/**/*.md`, owned HelixQA submodules' equivalents. Excludes: `external/**`, `prebuilts/**`, `packages/modules/**`, `kernel-5.10/**`, `out/**`, `build/**`, application/service source-code trees, and third-party submodules NOT in the owned set. Every edit triggers regeneration via `scripts/testing/sync_all_markdown_exports.sh` (pandoc HTML + weasyprint PDF, `timeout 60` per file, capped at 500 candidates). HTML + PDF mtime MUST be ≥ source `.md` mtime at all times.

Pre-build gates `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` + `CM-COVENANT-114-65-PROPAGATION`. Paired meta-test mutations. Composes with §11.4.12 / §11.4.18 / §11.4.23 / §11.4.44 / §11.4.45 /

§11.4.53 / §11.4.59 / §11.4.60 / §11.4.63 / §11.4.64. No escape hatch — no `--skip-md-exports`, `--no-pdf-only`, `--md-export-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.65.

Non-compliance is a release blocker regardless of context.

§11.4.66 — Blocker-resolution interactive-clarification mandate (User mandate, 2026-05-19)

When any task is blocked (operator decision, hardware access, external authorization, ambiguous scope), the agent MUST: (1) research what's doable from the agent side without operator input; (2) calculate minimum-viable operator input; (3) construct 2–4 mutually-exclusive options with one marked "Recommended" and each stating what the agent does after that answer; (4) present via the platform's interactive question mechanism (`AskUserQuestion` on Claude Code) — NEVER free-text "what would you like?" for closed- set decisions; (5) after the answer, resume work without follow-up round-trips. Composes with §11.4.6 / §11.4.7 / §11.4.40 / §11.4.41 / §11.4.42 / §11.4.52. No silent waiting; no bulk-text questions when interactive options would do.

Pre-build gate `CM-COVENANT-114-66-PROPAGATION` enforces the anchor literal across the 42-file consumer fleet. Paired meta- test mutation strips the literal → gate FAILs. No escape hatch — no `--skip-ask`, `--silent-wait`, `--free-form-only` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.66.

Non-compliance is a release blocker regardless of context.